

Cetus Vaults

Security Assessment

March 30th, 2024 — Prepared by OtterSec

Michał Bochnak embe221ed@osec.io

Andreas Mantzoutas andreas@osec.io

Robert Chen r@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

Findings **4**

Vulnerabilities **5**

OS-CPV-ADV-00 | Unchecked Role Removal 6

OS-CPV-ADV-01 | Incorrect Tick Range Expansion 7

OS-CPV-ADV-02 | Flash Loan Exploitation 8

General Findings **9**

OS-CPV-SUG-00 | Inconsistent Approach To Error Handling 10

OS-CPV-SUG-01 | Code Refactoring 11

OS-CPV-SUG-02 | Code Maturity 12

OS-CPV-SUG-03 | Absence Of Parameter Validation 13

Appendices

Vulnerability Rating Scale **14**

Procedure **15**

© 2024 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Cetus Protocol engaged OtterSec to assess the `vaults` program. This assessment was conducted between March 1st and March 20th, 2024. We conducted a follow-up review from May 27th to June 6th, 2025. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 7 findings throughout this audit engagement.

In particular, we identified a vulnerability, where the function neglects to verify if the member possesses the specified role before executing a subtraction operation, which may result in unexpected outcomes (OS-CPV-ADV-00). Additionally, it is possible to exploit the flash loan feature to repeatedly borrow funds, thereby reducing the repayment amount due to slippage adjustments, resulting in the unauthorized withdrawal of funds from the protocol (OS-CPV-ADV-02). Furthermore, migrating liquidity with a substantially larger tick range compared to the initial one may result in zero liquidity, rendering the vault unusable (OS-CPV-ADV-01).

We also made recommendations around adherence to coding best practices (OS-CPV-SUG-02) and suggested modifying the code base to enhance its security and efficiency (OS-CPV-SUG-01). Additionally, we advised adding parameter validation checks (OS-CPV-SUG-03).

© 2024 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/CetusProtocol/vaults. This audit was performed against commit c23ab34. We further reviewed the changes in PR#1, and 3eab21e in cetus compensation repo.

A brief description of the programs is as follows:

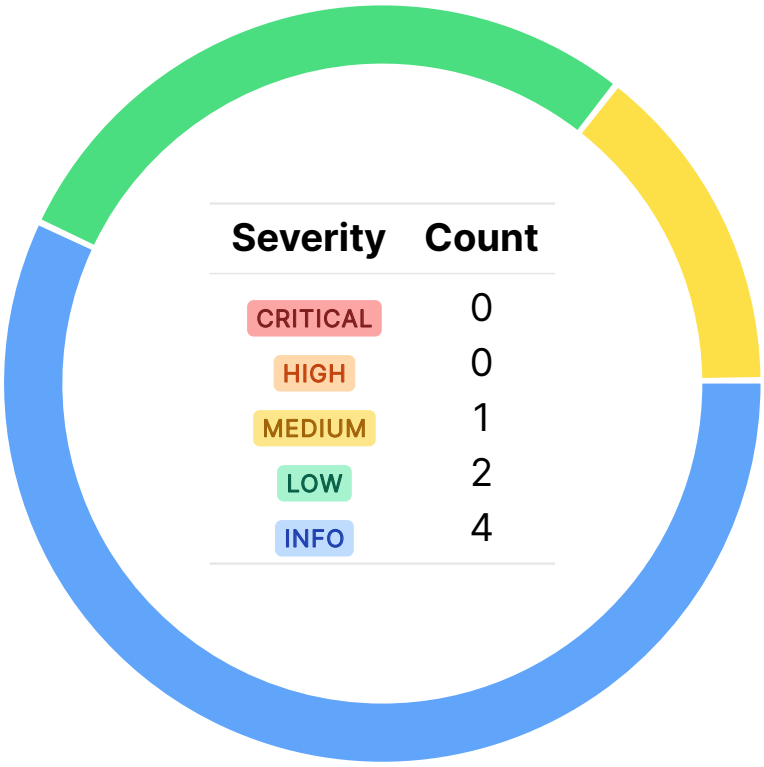
Name	Description
vaults	A system specifically designed to automatically manage user liquidity, encompassing the timely reinvestment of fees and rewards and rebalancing when necessary.

© 2024 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 7 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	0
MEDIUM	1
LOW	2
INFO	4

© 2024 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-CPV-ADV-00	MEDIUM	RESOLVED	`remove_role` neglects to verify if the member possesses the specified role before executing a subtraction operation, which may result in unexpected outcomes.
OS-CPV-ADV-01	LOW	RESOLVED	Calling `migrate_liquidity` with a substantially larger tick range compared to the initial one may result in zero liquidity, rendering the vault unusable.
OS-CPV-ADV-02	LOW	RESOLVED	It is possible to exploit the flash loan feature to repeatedly borrow funds, thereby reducing the repayment amount due to slippage adjustments, resulting in the unauthorized withdrawal of funds from the protocol.

© 2024 Otter Audits LLC. All Rights Reserved.

Unchecked Role Removal MEDIUM OS-CPV-ADV-00

Description

The current implementation of `acl::remove_role` directly subtracts the specified role from the member's permissions without initially verifying if the role is set for the member.

>_ *sources/acl.move* rust

```
/// @notice Revoke a role for a member in the ACL.
public fun remove_role(acl: &mut ACL, member: address, role: u8) {
    assert!(role < 128, ERoleNumberTooLarge);
    if (linked_table::contains(&acl.permissions, member)) {
        let perms = linked_table::borrow_mut(&mut acl.permissions, member);
        *perms = *perms - (1 << role);
    }
}
```

If the member's specified role is not set, the subtraction operation removes bits from the member's permissions that may correspond to other roles or unrelated data. This may result in unpredictable behavior and may inadvertently grant or revoke other roles for the member. Alternatively, if the subtraction operation attempts to subtract a role that is not set, it will set the role instead due to the two's complement arithmetic used in integer subtraction.

Remediation

Include a check to verify that the specified role is set for the member before executing the subtraction operation. Alternatively, modify the operation to utilize bit-wise `AND` with a bit mask that clears the bit corresponding to the specified role (`perms = *perms & (MAX_U128 - (1 << role))`). This guarantees that only the bit corresponding to the specified role is cleared, regardless of whether the role is currently set or not.

Patch

Fixed in 69ecf6d.

© 2024 Otter Audits LLC. All Rights Reserved.

Incorrect Tick Range Expansion LOW OS-CPV-ADV-01

Description

The vulnerability arises when `vaults::migrate_liquidity` is invoked with a significantly larger tick range (`[tick_lower; tick_upper]`) than the initial one. Here, `tick_lower` and `tick_upper` set the range of tick values for the new position, affecting the range of asset prices in the liquidity pool.

>_ *sources/vaults.move* rust

```
public fun migrate_liquidity<CoinTypeA, CoinTypeB, T>(
    manager: &VaultsManager,
    vault: &mut Vault<T>,
    [...]
    tick_lower: u32,
    tick_upper: u32,
    [...]
) {
    checked_package_version(manager);
    check_rebalance_role(manager, tx_context::sender(ctx));
    assert!(!vault.is_pause, EPoolIsPause);
    assert!(vault.status == STATUS_RUNNING, EVaultNotRunning);
    assert!(vector::length(&vault.positions) == 1, EPositionSizeError);

    // change status to `STATUS_REBALANCING`; After add remaining tokens into position will
    set the status to `STATUS_RUNNING`.
    vault.status = STATUS_REBALANCING;
    [...]
}
```

Expanding the tick range significantly may result in a scenario where the migrated liquidity results in zero liquidity. This occurs because the widened tick range may fail to effectively capture any trading or liquidity provision opportunities. Zero liquidity implies no assets are available for trading or providing liquidity in the pool. As a result, interacting with the vault or the liquidity pool becomes impossible or highly inefficient, rendering the vault unusable.

Remediation

Implement checks and limits to prevent excessive expansion beyond reasonable bounds.

Patch

Fixed in fb1586a.

© 2024 Otter Audits LLC. All Rights Reserved.

Flash Loan Exploitation LOW

OS-CPV-ADV-02

Description

The vulnerability stems from the interaction between `flash_loan` and `repay_flash_loan` within `vaults`, specifically in how the `repay_amount` is calculated and adjusted for slippage. Initially, the `repay_amount` is determined based on the original flash loan amount, with adjustments made for slippage. Consequently, if the operator borrows funds repeatedly to repay the previous loan, they may repay a reduced amount each time due to the slippage adjustment.

>_ *sources/vaults.move* rust

```
public fun flash_loan<CoinTypeA, CoinTypeB, T>(
    manager: &VaultsManager,
    vault: &mut Vault<T>,
    oracle_pool: &Pool<CoinTypeA, CoinTypeB>,
    amount: u64,
    loan_coin_a: bool,
    ctx: &mut TxContext,
): (Coin<CoinTypeA>, Coin<CoinTypeB>, FlashLoanReceipt) {
    [...]
    let repay_amount = repay_amount - repay_amount * (oracle_info.slippage as u64) /
        ↪  SLIPPAGE_DENOMINATOR;
    [...]
}
```

As the operator may borrow funds repeatedly, they may borrow the necessary amount to repay the previous loan. This establishes a loop where the operator continually borrows and repays loans, essentially siphoning funds from the protocol without contributing any external value. This presents a substantial risk to the protocol's financial integrity. Permitting operators to borrow and repay loans with reduced repayment amounts repeatedly will deplete the protocol's reserves over time.

Remediation

Ensure that loans are limited to only one coin type at a time to reduce complexity and potential abuse. By restricting loans to a single coin type, the protocol will have better control over the flow of funds and mitigate the risk of manipulative behavior.

Patch

Fixed in b7c15c9.

© 2024 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-CPV-SUG-00	`take_protocol_assets` aborts if the type name is not found, whereas `harvest_assets_amount` returns zero in the same scenario.
OS-CPV-SUG-01	Recommendations for modifying the code base to enhance its security and efficiency.
OS-CPV-SUG-02	Suggestions regarding ensuring adherence to best coding practices.
OS-CPV-SUG-03	The setters and `create_vault` do not verify the values of `max_quota` and `finish_rebalance_threshold`, allowing the pool manager to set arbitrary values.

© 2024 Otter Audits LLC. All Rights Reserved.

Inconsistent Approach To Error Handling OS-CPV-SUG-00

Description

In `Vaults`, `take_protocol_assets` and `harvest_assets_amount` handle non-existent type names differently. `take_protocol_assets` employs an assertion to halt execution promptly with an error message (`EInvalidCoinType`), while `harvest_assets_amount` returns zero, indicating the absence of assets available for harvesting of the specified `CoinType`.

Remediation

Adjust one function's implementation to align with the other to maintain consistency in the code base.

Patch

Resolved in 464c943.

© 2024 Otter Audits LLC. All Rights Reserved.

Code Refactoring OS-CPV-SUG-01

Description

1. `vaults::init` fails to utilize the `VERSION` constant when initializing the `VaultsManager`. The `VERSION` constant denotes the package version. Without referencing this version during the creation of `VaultsManager`, there exists a risk of inconsistency between the version indicated in the code and the actual version employed in the initialization process.

   >_ sources/vaults.move rust

   ```rust
   fun init(ctx: &mut TxContext) {
       [...]
       let manager = VaultsManager {
           id: object::new(ctx),
           index: 0,
           package_version: 1,
           vault_to_pool_maps: table::new(ctx),
           price_oracles: table::new(ctx),
           acl: acl::new(ctx)
       };
       [...]
   }
   ```

2. The current `docstring` for `vaults::add_oracle_pool` lacks information regarding the `slippage` parameter, essential for adding an oracle pool. This omission may confuse developers or users who depend on the documentation.

Remediation

1. Revise `vaults::init` to utilize the `VERSION` constant when creating the `VaultsManager` object.
2. Update the `docstring` to accurately reflect the current implementation of the function, including details about the `slippage` parameter.

Patch

1. Issue #1 resolved in 26b65a1.
2. Issue #2 resolved in b7c15c9.

© 2024 Otter Audits LLC. All Rights Reserved.

Code Maturity OS-CPV-SUG-02

Description

1. In `vaults::deposit`, substitute the literal value (`18446744073709551615u128`) with the named constant (`UINT64_MAX`) to improve code readability.
2. Both `acl::remove_role` and `acl::remove_member` fail to raise errors if the specified `member` does not exist; instead, they silently return. As a result, `vaults::remove_role` and `vaults::remove_member` emit an event even if no such action was taken.
3. Within `vaults`, `threshold` is misspelled as `thresold` in multiple instances.

   >_ sources/vaults.move rust

   ```rust
   /// Emit when update finish_rebalance_thresold
   struct UpdateRebalanceThresoldEvent has copy, drop {
       vault_id: ID,
       old_thresold: u64,
       new_thresold: u64
   }
   ```

4. Add an `is_pause` check to `vaults::harvest`, as it was removed from `harvest_internal`.
5. It is advisable to validate that the sum of `lp_amount` for a given `VaultVester` does not exceed its `total_supply`.

Remediation

Implement the above-mentioned suggestions.

Patch

1. Issue #1 resolved in b7c15c9.
2. Issue #2 resolved in 26b65a1.
3. Issue #3 resolved in 26b65a1.
4. Issue #4 resolved in 21a40f3.
5. Issue #5 resolved in 3b777dd.

© 2024 Otter Audits LLC. All Rights Reserved.

Absence Of Parameter Validation OS-CPV-SUG-03

Description

The vulnerability stems from the absence of validation for the `max_quota` and `finish_rebalance_threshold` values in their setter functions or in `create_vault` within `vault`. Without proper validation, the pool manager may assign arbitrary and potentially unintended values to `max_quota` and `finish_rebalance_threshold`.

>_ sources/vaults.move rust

```
public fun create_vault<CoinTypeA, CoinTypeB, T>(
     [...]
     max_quota: u128,
     finish_rebalance_threshold: u64,
     clk: &Clock,
     ctx: &mut TxContext
) {
    checked_package_version(manager);
    check_pool_manager_role(manager, tx_context::sender(ctx));
    assert!(total_supply<T>(&lp_token_treasury) == 0, ETreausyCapIllegal);
    [...]
}
```

Assigning arbitrary values to these parameters may result in instability within the system. A high `max_quota` may cause unexpected resource allocation or imbalances in the vaults system. Similarly, setting inappropriate values for `finish_rebalance_threshold` may disrupt the re-balancing process and impact the system's overall performance.

Remediation

Implement validation checks for `max_quota` and `finish_rebalance_threshold`.

© 2024 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2024 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that the other missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2024 Otter Audits LLC. All Rights Reserved.